NurdleDNA — Judge Q&A Cheat Sheet

ECTE 250 · Team 2 (Arc Tech) · University of Wollongong in Dubai — lead with the headline number; keep each answer ~20–30 s.

~3 s detection | $232 → $180 / 100 units | 0 false alarms (30 min) | 230,000 t/yr to oceans | 19 AI classes | valve close ~300 ms | ~15 W

Problem & Motivation

What is a nurdle? 2–5 mm plastic pellets, the raw material for all plastics; the **2nd-largest** primary ocean microplastic source (~230,000 t/yr).

Why the UAE / why now? Polymer & petrochemical hub (Borouge, ADNOC) → high spill risk. cf. the X-Press Pearl disaster — the largest nurdle spill in history.

Who is the customer? Port operators, polymer plants, pellet-handling facilities — any industrial outfall or drain stack.

AI & Computer Vision — expect deep questions here

Which model, and why? YOLOv8n — the smallest YOLOv8 (~3M params), real-time on the edge; exported to **ONNX** so it runs on the Jetson via onnxruntime with no PyTorch/Ultralytics on-device (~100 MB deps).

Walk me through the inference pipeline. Frame → resize 640×640 → ONNX inference → NMS (IoU 0.45, conf 0.25) → central-60% ROI mask → class filter → count → EMA smoothing → 2.5 s confirmation → FSM state.

Why ONNX, not PyTorch or TensorRT? ONNX runtime is lightweight and portable — full torch won't install cleanly on a Jetson Nano. TensorRT is our next optimization for higher FPS.

How accurate is it? Per-class mAP@50: Pen 0.81, Air-bubble 0.75, Microfibre 0.72, Fragment 0.60. Overall mAP is lower because it spans 19 classes — we cite the relevant per-class figures.

What does mAP@50 mean? Mean Average Precision at IoU 0.50 — how well predicted boxes match ground truth across classes; the standard object-detection metric.

How do you avoid FALSE POSITIVES? Hybrid YOLO + HSV plus four filters: ROI mask, circularity filter, EMA smoothing, and a 2.5 s confirmation timer; background/air-bubble classes excluded → **0 false alarms in 30 min**.

What about FALSE NEGATIVES (missed pellets)? HSV white-blob detection catches pellets YOLO misses; gas/turbidity sensors are a second line; confidence threshold tuned for sensitivity.

Edge or cloud inference? 100% on-device — no cloud dependency for detection. Low latency, works offline, data stays local.

Training details? Transfer-learned YOLOv8n on Roboflow microplastics-t0ddd v6 (3,102 images, 19 classes), 25 epochs on a Colab T4 GPU.

Why not classical CV alone? HSV finds white blobs but can't classify type or handle varied appearance; YOLO classifies. Hybrid = recall of CV + intelligence of the CNN.

Changing lighting / murky water? Central ROI + tuned HSV + the confirmation timer reject glare and flicker; a UV LED gives controlled illumination; the turbidity sensor cross-checks.

How would you improve accuracy? More diverse field data + active learning (retrain on captured proof frames), TensorRT, higher-res ROI crops, and consolidating the 19 classes.

Does it learn / drift over time? Every alarm saves a proof image to the cloud — a labelled feedback dataset for periodic retraining.

What is the 'Density Index'? A 0–100 score derived from the live pellet count — one at-a-glance contamination level for the dashboard.

Could you run a bigger model? Yes (YOLOv8s/m) for accuracy, but it costs FPS on the Nano; nano was chosen for real-time edge inference.

Electronics, FSM & Safety

Why two processors? Jetson = AI vision; the **Arduino owns the valve** via a deterministic 5-state Moore FSM. If the Jetson fails, the Arduino still alarms from analog sensors — safety-critical separation.

Explain the FSM. S0 Init → S1 OK → S2 Caution → S3 ALARM (**latched**) → reset. Latched so contamination can't silently clear; reset by operator button or remote dashboard.

Response time? ~3 s detection end-to-end; valve closure ~300 ms.

How do the boards communicate? JSON over USB serial at 5 Hz, with strict parsing to reject noise.

Testing, Cost & Scale

How did you test it? 30-min bench test (varied lighting, no pellets → 0 false alarms); redundancy test (Jetson unplugged → Arduino still alarmed); measured latency & valve timing.

Does it work in real conditions? Bench-validated today; field trials with real flow and turbidity are the next step (be honest).

Cost & scale? $232 prototype → ~$180 at 100 units; ~15 W, ~$15/yr power. Each unit has a unique Firebase ID; a port equips 4–10 bays for less than one annual lab contract.

What's next? Oil-droplet detection (retrain the model) and a UV-fluorescence upgrade for forensic source-tracing.

Sustainability & Hard Questions

Which SDGs? 14 Life Below Water, 9 Industry/Innovation, 6 Clean Water, 12 Responsible Consumption.

Real-world impact? Stops contamination **at source**, before it reaches the Gulf — prevention with an audit trail, not after-the-fact cleanup.

Prove an event to a regulator? Immutable Firebase log: timestamp, captured mass, gas, density index, bay ID — searchable & exportable.

Why not just filter everything? Filters clog and can't identify type, log evidence, or prove compliance — you lose the audit trail.

What are the limitations? Bench- not field-validated; AI depends on water clarity/lighting; the cartridge needs periodic maintenance.

> **⚠ Be honest: live vs designed (pre-agree your wording)**
> **Live now:** camera + YOLO detection, gas sensor, FSM + LED states, Firebase dashboard, remote reset.
> **Designed / shown in the digital twin (not on the current breadboard):** servo pinch valve (removed from the board — the valve *logic* is live and shown on the dashboard/twin), load-cell mass capture, LDR turbidity. Say "the control logic drives it — here it is on the twin," never claim a physical part actuates if it isn't wired.